Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 4

«Запросы на выборку и модификацию данных. Представления. Работа с индексами.»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся: Лукошников Дмитрий Викторович
Факультет: прикладной информатики
Группа: K3242
Направление подготовки: 09.03.03 Прикладная информатика
Образовательная программа: Мобильные и сетевые технологии
Преподаватели: Говорова Марина Михайловна
Белов Александр

Санкт-Петербург

2026 г.

1 Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2 Практическое задание

В рамках лабораторной работы требовалось:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3);
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов;
3. Изучить графическое представление запросов и просмотреть историю запросов;
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN ANALYZE.

3 Выполнение работы

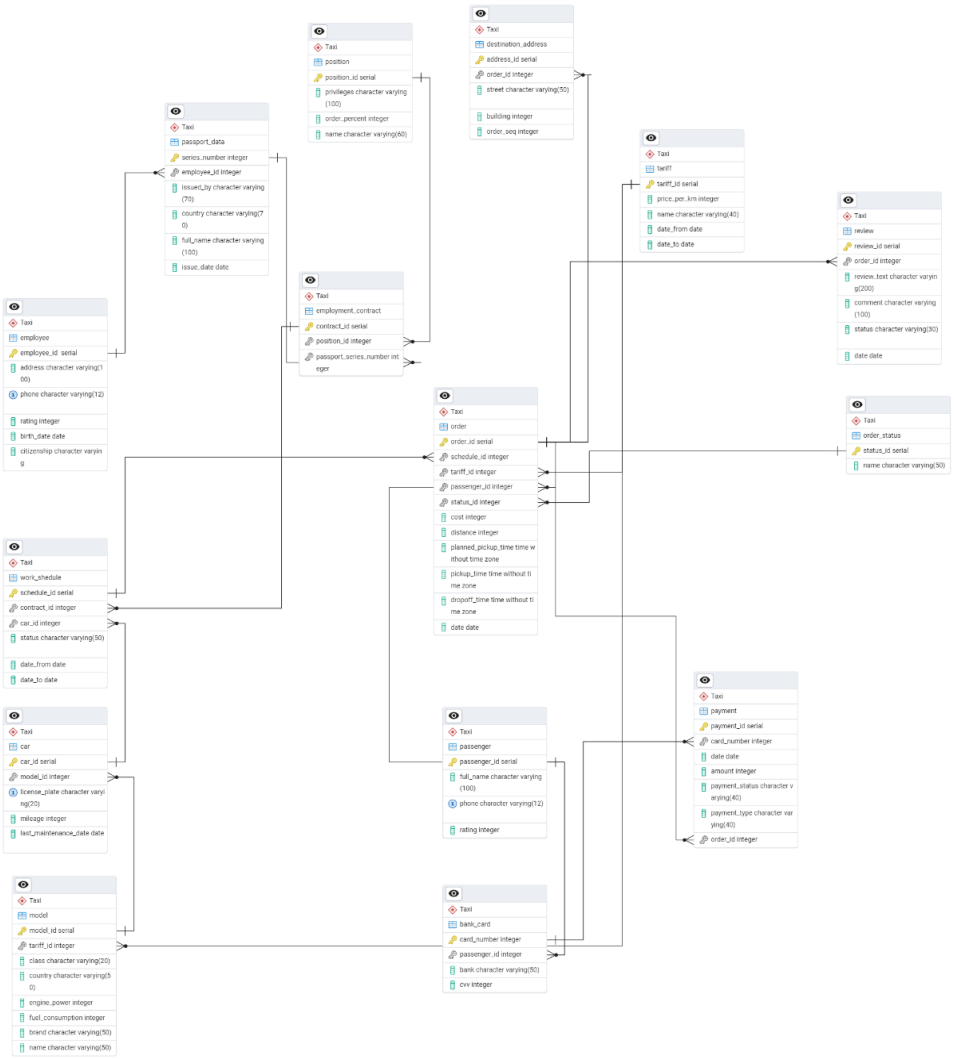
3.1 Наименование базы данных и вариант

Вариант: 1 — БД «**Служба заказа такси**».

Наименование базы данных: TaxiOrder.

Схема базы данных: Taxi.

Схема базы данных ERD:



3.2 Запросы на выборку данных

Запрос 1. Вывести данные о водителе, который чаще всех доставляет пассажиров на заданную улицу.

Запрос и результат выполнения запроса 1

Query Query History

```
SELECT
e.employee_id,
pd.full_name,
e.phone,
e.rating,
COUNT(o.order_id) AS "Количество поездок на ул. Тверская"
FROM "Taxi".employee e
JOIN "Taxi".passport_data pd ON e.employee_id = pd.employee_id
JOIN "Taxi".employment_contract ec ON pd.series_number = ec.passport_series_number
JOIN "Taxi"."position" pos ON ec.position_id = pos.position_id
JOIN "Taxi".work_shedule ws ON ec.contract_id = ws.contract_id
JOIN "Taxi"."order" o ON ws.schedule_id = o.schedule_id
JOIN "Taxi".destination_address da ON o.order_id = da.order_id
WHERE pos.name = 'водитель' AND da.street = 'ул. Тверская'
GROUP BY e.employee_id, pd.full_name, e.phone, e.rating;
```

Data Output Messages Explain Notifications

	employee_id integer	full_name character varying (100)	phone character varying (12)	rating integer	Количество поездок на ул. Тверская bigint
1	1	Сергеев Сергей Петрович	+79031234567	5	2
2	5	Кузнецов Андрей Сергеевич	+79034445566	4	1

Запрос 2. Вывести данные об автомобилях, которые имеют пробег более 250 тысяч. километров и которые не проходили ТО в текущем году.

Выполнение запроса 2

Query Query History

```
SELECT
c.car_id,
c.license_plate,
c.mileage,
c.last_maintenance_date,
m.brand,
m.name
FROM "Taxi".car c
JOIN "Taxi".model m ON c.model_id = m.model_id
WHERE c.mileage > 250000
AND EXTRACT(YEAR FROM c.last_maintenance_date) < EXTRACT(YEAR FROM CURRENT_DATE);
```

Data Output Messages Explain Notifications

	car_id integer	license_plate character varying (20)	mileage integer	last_maintenance_date date	brand character varying (50)	name character varying (50)
1	13	М789НН777	312000	2023-10-15	Kia	Rio
2	14	Н012ПП777	278500	2023-12-01	Mercedes-Benz	E-Class
3	15	О345РР777	405000	2023-08-20	ГАЗ	Газель Next

Запрос 3. Сколько раз каждый пассажир воспользовался услугами таксопарка?

Выполнение запроса 3

Query Query History

```
1 SELECT
2 p.passenger_id,
3 p.full_name,
4 p.phone,
5 p.rating,
6 COUNT(o.order_id) AS "Количество поездок"
7 FROM "Taxi".passenger p
8 LEFT JOIN "Taxi"."order" o ON p.passenger_id = o.passenger_id
9 GROUP BY p.passenger_id;
```

Data Output Messages Explain ✕ Notifications

	passenger_id [PK] integer	full_name character varying (100)	phone character varying (12)	rating integer	Количество поездок bigint
1	5	Михайлов Дмитрий Андреевич	+79165556677	5	2
2	4	Козлова Елена Владимировна	+79164445566	4	2
3	10	Дмитриева Наталья Павловна	+79160001122	4	1
4	6	Федорова Ольга Николаевна	+79166667788	4	2
5	13	Новичков Николай Нетзаказов	+79990001122	5	0
6	2	Петрова Мария Сергеевна	+79162223344	5	2
7	7	Николаев Павел Сергеевич	+79167778899	3	1
8	1	Иванов Иван Иванович	+79161112233	4	2
9	8	Алексеева Анна Викторовна	+79168889900	5	1
10	11	Андреев Максим Романович	+79169991122	4	1
11	9	Степанов Артем Игоревич	+79169990011	4	1
12	3	Сидоров Алексей Петрович	+79163334455	3	2
13	12	Романов Виктор Викторович	+79162227788	5	1

Запрос 4. Вывести данные пассажира, который воспользовался услугами таксопарка максимальное число раз.

Выполнение запроса 4

Query Query History

```
1  SELECT
2  p.passenger_id,
3  p.full_name,
4  p.phone,
5  p.rating,
6  COUNT(o.order_id) AS "Количество поездок"
7  FROM "Taxi".passenger p
8  JOIN "Taxi"."order" o ON p.passenger_id = o.passenger_id
9  GROUP BY p.passenger_id
10 HAVING COUNT(o.order_id) = (SELECT MAX(order_count)
11 FROM (SELECT COUNT(order_id) AS order_count
12 FROM "Taxi"."order"
13 GROUP BY passenger_id));
```

Data Output Messages Explain Notifications

	passenger_id [PK] integer	full_name character varying (100)	phone character varying (12)	rating integer	Количество поездок bigint
1	4	Козлова Елена Владимировна	+79164445566	4	2

В этом запросе может вывести несколько пассажиров, так как у них одинаково максимальное количество поездок. Если нужен один из них можно добавить LIMIT 1.

Query Query History

```sql
SELECT
p.passenger_id,
p.full_name,
p.phone,
p.rating,
COUNT(o.order_id) AS "Количество поездок"
FROM "Taxi".passenger p
JOIN "Taxi"."order" o ON p.passenger_id = o.passenger_id
GROUP BY p.passenger_id
HAVING COUNT(o.order_id) = (SELECT MAX(order_count)
FROM (SELECT COUNT(order_id) AS order_count
FROM "Taxi"."order"
GROUP BY passenger_id)) LIMIT 1;
```

Data Output Messages Explain × Notifications

	passenger_id [PK] integer	full_name character varying (100)	phone character varying (12)	rating integer	Количество поездок bigint
1	4	Козлова Елена Владимировна	+79164445566	4	2

Запрос 5. Вывести данные о водителе, который ездит на самом дорогом автомобиле.

Выполнение запроса 5

Query Query History

```sql
SELECT
e.employee_id,
pd.full_name,
e.phone,
e.rating,
c.license_plate,
c.price
FROM "Taxi".employee e
JOIN "Taxi".passport_data pd ON e.employee_id = pd.employee_id
JOIN "Taxi".employment_contract ec ON pd.series_number = ec.passport_series_number
JOIN "Taxi"."position" pos ON ec.position_id = pos.position_id
JOIN "Taxi".work_shedule ws ON ec.contract_id = ws.contract_id
JOIN "Taxi".car c ON ws.car_id = c.car_id
WHERE pos.name = 'водитель' AND c.price = (SELECT MAX(price) FROM "Taxi".car);
```

Data Output Messages Explain × Notifications

	employee_id integer	full_name character varying (100)	phone character varying (12)	rating integer	license_plate character varying (20)	price integer
1	5	Кузнецов Андрей Сергеевич	+79034445566	4	Н012ПП777	14000000

Запрос 6. Вывести данные пассажира, который всегда ездит с одним и тем же водителем.

Выполнение запроса 6

Query Query History

```sql
SELECT
p.passenger_id,
p.full_name,
p.phone,
p.rating
FROM "Taxi".passenger p
WHERE (SELECT COUNT(DISTINCT ec.contract_id)
FROM "Taxi"."order" o
JOIN "Taxi".work_shedule ws ON o.schedule_id = ws.schedule_id
JOIN "Taxi".employment_contract ec ON ws.contract_id = ec.contract_id
JOIN "Taxi"."position" p1 ON ec.position_id = p1.position_id
WHERE o.passenger_id = p.passenger_id and p1.name = 'водитель') = 1;
```

Data Output Messages Explain × Notifications

	passenger_id [PK] integer	full_name character varying (100)	phone character varying (12)	rating integer
1	1	Иванов Иван Иванович	+79161112233	4
2	2	Петрова Мария Сергеевна	+79162223344	5
3	3	Сидоров Алексей Петрович	+79163334455	3
4	4	Козлова Елена Владимировна	+79164445566	4
5	5	Михайлов Дмитрий Андреевич	+79165556677	5
6	6	Федорова Ольга Николаевна	+79166667788	4
7	8	Алексеева Анна Викторовна	+79168889900	5
8	9	Степанов Артем Игоревич	+79169990011	4
9	12	Романов Виктор Викторович	+79162227788	5

Запрос 7. Какие автомобили имеют пробег больше среднего пробега для своей марки

Выполнение запроса 7

Query Query History

```
1  SELECT
2  c.car_id,
3  c.license_plate,
4  m.brand,
5  c.mileage,
6  (SELECT ROUND(AVG(c2.mileage), 0)
7  FROM "Taxi".car c2
8  JOIN "Taxi".model m2 ON c2.model_id = m2.model_id
9  WHERE m2.brand = m.brand) AS "Средний пробег по марке"
10 FROM "Taxi".car c
11 JOIN "Taxi".model m ON c.model_id = m.model_id
12 WHERE c.mileage > (
13 SELECT AVG(c2.mileage)
14 FROM "Taxi".car c2
15 JOIN "Taxi".model m2 ON c2.model_id = m2.model_id
16 WHERE m2.brand = m.brand);
```

Data Output Messages Explain Notifications

	car_id integer	license_plate character varying (20)	brand character varying (50)	mileage integer	Средний пробег по марке numeric
1	2	Б456ГД777	Lada	67800	56500
2	11	К123ЦЧ777	Lada	56700	56500
3	12	Л456ШЩ777	Hyundai	89000	56200
4	13	М789НН777	Kia	312000	200600
5	14	Н012ПП777	Mercedes-Benz	278500	167600
6	15	О345РР777	ГАЗ	405000	258650

3.3 Создание представлений

Представление 1. содержащее сведения о незанятых на данный момент водителях;

Создание представления free_drivers_view

Query Query History

```sql
CREATE OR REPLACE VIEW "Taxi".free_drivers_view AS
SELECT
e.employee_id AS "ID сотрудника",
pd.full_name AS "ФИО водителя",
e.phone AS "Телефон",
e.rating AS "Рейтинг",
CASE
WHEN ws.schedule_id IS NULL THEN 'Не в смене'
WHEN ws.status = 'не начат' THEN 'Смена не начата'
WHEN ws.status = 'закончен' THEN 'Смена закончена'
ELSE 'В смене, но без активных заказов'
END AS "Статус"
FROM "Taxi".employee e
JOIN "Taxi".passport_data pd ON e.employee_id = pd.employee_id
JOIN "Taxi".employment_contract ec ON pd.series_number = ec.passport_series_number
JOIN "Taxi"."position" pos ON ec.position_id = pos.position_id
JOIN "Taxi".work_shedule ws ON ec.contract_id = ws.contract_id
AND ws.date_from <= CURRENT_DATE
AND (ws.date_to IS NULL OR ws.date_to >= CURRENT_DATE)
JOIN "Taxi"."order" o ON ws.schedule_id = o.schedule_id
AND o.date = CURRENT_DATE
AND o.status_id != (SELECT status_id FROM "Taxi".order_status WHERE name = 'закончен')
WHERE pos.name = 'водитель'
GROUP BY e.employee_id, pd.full_name, e.phone, e.rating, ws.schedule_id, ws.status
HAVING COUNT(o.order_id) = 0 OR ws.schedule_id IS NULL;
```

Query Query History

```sql
select * from "Taxi".free_drivers_view;
```

Data Output Messages Explain Notifications

	employee_id integer	full_name character varying (100)	phone character varying (12)	rating integer	Статус text
1	1	Сергеев Сергей Петрович	+79031234567	5	В смене, но без активных заказов
2	3	Иванов Дмитрий Алексеевич	+79039876543	5	В смене, но без активных заказов
3	5	Кузнецов Андрей Сергеевич	+79034445566	4	В смене, но без активных заказов
4	8	Лебедева Татьяна Николаевна	+79036667788	5	В смене, но без активных заказов
5	1	Сергеев Сергей Петрович	+79031234567	5	Смена не начата
6	5	Кузнецов Андрей Сергеевич	+79034445566	4	В смене, но без активных заказов

Представление 2. зарплата всех водителей за вчерашний день

Создание представления drive_salary_yesterday_view

Query Query History

```sql
CREATE OR REPLACE VIEW "Taxi".driver_salary_yesterday_view AS
SELECT
e.employee_id,
pd.full_name,
e.phone,
e.rating,
ROUND(COALESCE(SUM(o.cost), 0) * pos.order_percent / 100, 2) AS "Зарплата за вчера (руб.)"
FROM "Taxi".employee e
JOIN "Taxi".passport_data pd ON e.employee_id = pd.employee_id
JOIN "Taxi".employment_contract ec ON pd.series_number = ec.passport_series_number
JOIN "Taxi"."position" pos ON ec.position_id = pos.position_id
JOIN "Taxi".work_shedule ws ON ec.contract_id = ws.contract_id
LEFT JOIN "Taxi"."order" o ON ws.schedule_id = o.schedule_id
AND o.date = CURRENT_DATE - INTERVAL '1 day'
WHERE pos.name = 'водитель'
GROUP BY e.employee_id, pd.full_name, e.phone, pos.order_percent
ORDER BY "Зарплата за вчера (руб.)" DESC;
```

Query Query History

```sql
select * from "Taxi".driver_salary_yesterday_view;
```

Data Output Messages Explain Notifications

	employee_id integer	full_name character varying (100)	phone character varying (12)	rating integer	Зарплата за вчера (руб.) numeric
1	1	Сергеев Сергей Петрович	+79031234567	5	35.00
2	3	Иванов Дмитрий Алексеевич	+79039876543	5	0.00
3	5	Кузнецов Андрей Сергеевич	+79034445566	4	0.00
4	8	Лебедева Татьяна Николаевна	+79036667788	5	0.00

3.4 Запросы на модификацию данных с использованием подзапросов

Запрос INSERT

Формулировка: добавить данные о банковской карте существующему пассажиру - Иванову Ивану Ивановичу.

Данные до

Query Query History

```
SELECT * FROM "Taxi".bank_card WHERE passenger_id = (SELECT passenger_id FROM "Taxi".passenger WHERE full_name = 'Иванов Иван Иванович')
```

Data Output Messages Explain Notifications

Showing rows: 1 to 1 Page N

	card_number [PK] integer	passenger_id integer	bank character varying (50)	cvv integer	year_to integer	month_to integer
1	12345678	1	Сбербанк	123	26	12

Выполнение запроса

Query Query History

```
INSERT INTO "Taxi".bank_card (card_number, passenger_id, bank, cvv, year_to, month_to)
VALUES (99887766,
(SELECT passenger_id FROM "Taxi".passenger WHERE full_name = 'Иванов Иван Иванович'),
'Т-Банк', 777, 28, 12);
```

Проверка вставки данных

```
select * from "Taxi".bank_card
where passenger_id = (select passenger_id from "Taxi".passenger where full_name = 'Иванов Иван Иванович');
```

Data Output Messages Explain Notifications

	card_number [PK] integer	passenger_id integer	bank character varying (50)	cvv integer	year_to integer	month_to integer
1	12345678	1	Сбербанк	123	26	12
2	99887766	1	Т-Банк	777	28	12

Запрос UPDATE

Формулировка: установить статус заказа – закончен, у заказов, которые уже в прошлом(дата заказа < текущей даты).

Данные до

Query Query History

```
select * from "Taxi".order;
```

Data Output Messages Explain Notifications

Showing rows: 1 to 18 Pag

	order_id [PK] integer	schedule_id integer	tariff_id integer	passenger_id integer	status_id integer	cost integer	distance integer	planned_pickup_time time without time zone	pickup_time time without time zone	dropoff_time time without time zone	date date
1	3	3	1	3	3	320	11	14:00:00	14:02:00	14:30:00	2024-10-02
2	4	4	2	4	3	560	12	15:45:00	15:50:00	16:25:00	2024-10-02
3	5	5	3	5	3	890	12	09:15:00	09:18:00	09:55:00	2024-10-03
4	6	6	1	6	3	430	15	11:30:00	11:35:00	12:10:00	2024-10-03
5	7	7	1	7	3	1240	16	17:00:00	17:10:00	[null]	2024-10-04
6	8	8	2	8	3	670	14	18:30:00	18:32:00	19:15:00	2024-10-04
7	9	9	3	9	3	350	5	20:00:00	[null]	[null]	2024-10-05
8	10	10	4	10	3	520	6	08:45:00	08:47:00	09:20:00	2024-10-05
9	11	11	2	11	3	980	21	13:15:00	13:20:00	14:10:00	2024-10-06
10	12	12	1	12	3	430	15	16:45:00	16:48:00	17:25:00	2024-10-06
11	13	1	2	1	3	670	14	11:00:00	11:02:00	11:45:00	2024-10-07
12	14	3	3	2	3	890	12	15:30:00	15:35:00	16:10:00	2024-10-07
13	15	4	1	3	3	500	18	09:00:00	09:05:00	09:40:00	2024-10-10
14	16	5	2	4	3	600	15	10:30:00	10:35:00	11:10:00	2024-10-10
15	17	6	1	5	3	450	12	12:00:00	12:03:00	12:35:00	2024-10-11
16	18	1	1	6	3	550	16	14:00:00	14:05:00	14:45:00	2024-10-11
17	1	1	1	1	1	450	15	10:00:00	10:05:00	10:35:00	2024-10-01
18	2	2	2	2	3	780	16	12:30:00	12:35:00	13:15:00	2026-04-17

Выполнение запроса

Query Query History

```
UPDATE "Taxi"."order"
SET status_id = (SELECT status_id FROM "Taxi".order_status WHERE name = 'закончен')
WHERE date < CURRENT_DATE;
```

Проверка

Query Query History

```
select * from "Taxi".order;
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 18

	order_id [PK] integer	schedule_id integer	tariff_id integer	passenger_id integer	status_id integer	cost integer	distance integer	planned_pickup_time time without time zone	pickup_time time without time zone	dropoff_time time without time zone	date date
1	2	2	2	2	3	780	16	12:30:00	12:35:00	13:15:00	2026-04-17
2	3	3	1	3	3	320	11	14:00:00	14:02:00	14:30:00	2024-10-02
3	4	4	2	4	3	560	12	15:45:00	15:50:00	16:25:00	2024-10-02
4	5	5	3	5	3	890	12	09:15:00	09:18:00	09:55:00	2024-10-03
5	6	6	1	6	3	430	15	11:30:00	11:35:00	12:10:00	2024-10-03
6	7	7	1	7	3	1240	16	17:00:00	17:10:00	[null]	2024-10-04
7	8	8	2	8	3	670	14	18:30:00	18:32:00	19:15:00	2024-10-04
8	9	9	3	9	3	350	5	20:00:00	[null]	[null]	2024-10-05
9	10	10	4	10	3	520	6	08:45:00	08:47:00	09:20:00	2024-10-05
10	11	11	2	11	3	980	21	13:15:00	13:20:00	14:10:00	2024-10-06
11	12	12	1	12	3	430	15	16:45:00	16:48:00	17:25:00	2024-10-06
12	13	1	2	1	3	670	14	11:00:00	11:02:00	11:45:00	2024-10-07
13	14	3	3	2	3	890	12	15:30:00	15:35:00	16:10:00	2024-10-07
14	15	4	1	3	3	500	18	09:00:00	09:05:00	09:40:00	2024-10-10
15	16	5	2	4	3	600	15	10:30:00	10:35:00	11:10:00	2024-10-10
16	17	6	1	5	3	450	12	12:00:00	12:03:00	12:35:00	2024-10-11
17	18	1	1	6	3	550	16	14:00:00	14:05:00	14:45:00	2024-10-11
18	1	1	1	1	3	450	15	10:00:00	10:05:00	10:35:00	2024-10-01

Запрос DELETE

Формулировка: удалить комментарии у еще не законченных поездок.

Данные до

```
select * from "Taxi".review;
```

Data Output Messages Explain × Notifications

	review_id [PK] integer	order_id integer	review_text character varying (200)	comment character varying (100)	status character varying (30)	date date
1	1	1	Отличная поездка, вежливый водитель	Всё понравилось	опубликован	2024-10-01
2	2	2	Машина чистая, приехали вовремя	Рекомендую	опубликован	2024-10-01
3	3	3	Хорошо, но немного опоздали	Могли бы и побыстрее	опубликован	2024-10-02
4	4	4	[null]	Без комментариев	отклонен	2024-10-03
5	5	5	Прекрасный сервис	Спасибо!	опубликован	2024-10-04
6	6	6	Нормально	[null]	опубликован	2024-10-05
7	7	7	Водитель профессионал	[null]	опубликован	2024-10-06

```
select * from "Taxi".order
where status_id = 1;
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 1

	order_id [PK] integer	schedule_id integer	tariff_id integer	passenger_id integer	status_id integer	cost integer	distance integer	planned_pickup_time time without time zone	pickup_time time without time zone	dropoff_time time without time zone	date date
1	5	5	3	5	1	890	12	09:15:00	09:18:00	09:55:00	2024-10-03

Запрос

Query Query History

```
1 DELETE FROM "Taxi".review
2 WHERE order_id IN (SELECT order_id
3 FROM "Taxi"."order"
4 WHERE status_id = (SELECT status_id FROM "Taxi".order_status WHERE name = 'не закончен'));
```

Проверка

Data Output Messages Explain Notifications

	review_id [PK] integer	order_id integer	review_text character varying (200)	comment character varying (100)	status character varying (30)	date date
1	1	1	Отличная поездка, вежливый водитель	Всё понравилось	опубликован	2024-10-01
2	2	2	Машина чистая, приехали вовремя	Рекомендую	опубликован	2024-10-01
3	3	3	Хорошо, но немного опоздали	Могли бы и побыстрее	опубликован	2024-10-02
4	4	4	[null]	Без комментариев	отклонен	2024-10-03
5	6	6	Нормально	[null]	опубликован	2024-10-05
6	7	7	Водитель профессионал	[null]	опубликован	2024-10-06

3.5 Создание индексов и анализ плана выполнения запросов

Для анализа влияния индексов на выполнение запросов были выбраны два запроса. Для каждого был получен план выполнения и время работы до создания индекса, после чего был создан индекс и выполнен повторный анализ.

Простой индекс для даты

Для анализа был выбран запрос найти все поездки в промежутке между '2024-10-01' и '2024-10-05'

План выполнения запроса до создания простого индекса

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM "Taxi"."order"
3  WHERE date BETWEEN '2024-10-01' AND '2024-10-05';
```

Data Output Messages Explain × Notifications

	QUERY PLAN text
1	Seq Scan on "order" (cost=0.00..1.27 rows=10 width=56) (actual time=0.019..0.022 rows=10 loops=...
2	Filter: ((date >= '2024-10-01'::date) AND (date <= '2024-10-05'::date))
3	Rows Removed by Filter: 8
4	Planning Time: 0.091 ms
5	Execution Time: 0.197 ms

Для ускорения выполнения запроса был создан простой индекс по полю date таблицы order.

Создание простого индекса

Query Query History

```
1  CREATE INDEX idx_order_date ON "Taxi"."order" (date);
```

После создания индекса запрос был повторно проанализирован.

План выполнения запроса после создания простого индекса

Query Query History

Execute script
F5

```
1 EXPLAIN ANALYZE
2 SELECT * FROM "Taxi"."order"
3 WHERE date BETWEEN '2024-10-01' AND '2024-10-05';
```

Data Output Messages Explain × Notifications

SQL

	QUERY PLAN text
1	Seq Scan on "order" (cost=0.00..1.27 rows=10 width=56) (actual time=0.033..0.037 rows=10 loops=...
2	Filter: ((date >= '2024-10-01'::date) AND (date <= '2024-10-05'::date))
3	Rows Removed by Filter: 8
4	Planning Time: 2.495 ms
5	Execution Time: 0.059 ms

Запрос выполнился быстрее с индексом

Составной индекс

До создания индекса для запроса был получен план выполнения.

План выполнения запроса до создания составного индекса

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM "Taxi"."order"
3  WHERE status_id = 3
4  AND date BETWEEN '2024-10-01' AND '2024-10-05';
```

Data Output Messages Explain × Notifications

	QUERY PLAN text
1	Seq Scan on "order" (cost=0.00..1.31 rows=10 width=56) (actual time=0.033..0.038 rows=10 loops=...
2	Filter: ((date >= '2024-10-01'::date) AND (date <= '2024-10-05'::date) AND (status_id = 3))
3	Rows Removed by Filter: 8
4	Planning Time: 0.143 ms
5	Execution Time: 0.065 ms

Для данного запроса был создан составной индекс по полям date и status_id таблицы order.

Создание составного индекса

Query Query History

```
1  CREATE INDEX idx_order_status_date ON "Taxi"."order" (status_id, date);
```

После этого был повторно получен план выполнения запроса.

Query Query History

Execute script F5

```
1  EXPLAIN ANALYZE
2  SELECT * FROM "Taxi"."order"
3  WHERE status_id = 3
4  AND date BETWEEN '2024-10-01' AND '2024-10-05';
```

Data Output Messages Explain × Notifications

	QUERY PLAN text
1	Seq Scan on "order" (cost=0.00..1.31 rows=10 width=56) (actual time=0.022..0.026 rows=10 loops=...
2	Filter: ((date >= '2024-10-01'::date) AND (date <= '2024-10-05'::date) AND (status_id = 3))
3	Rows Removed by Filter: 8
4	Planning Time: 2.472 ms
5	Execution Time: 0.046 ms

Запрос с индексом отработал быстрее

Все индексы были удалены

Query Query History

```
1  drop index "Taxi".idx_order_status_date;
2  drop index "Taxi".idx_order_date;
3
```

4 Выводы

В ходе выполнения лабораторной работы были успешно освоены практические навыки создания сложных запросов на выборку данных с использованием многотабличных соединений, агрегатных функций и подзапросов, а также разработаны представления для удобства работы с часто используемыми выборками, включая информацию о свободных водителях и расчёте заработной платы. Были реализованы запросы на модификацию данных (INSERT, UPDATE, DELETE) с применением подзапросов для обеспечения их точности и гибкости. Для оптимизации производительности созданы простой и составной индексы, выполнено сравнение времени выполнения запросов без индексов и с индексами с использованием команды EXPLAIN ANALYZE, что позволило наглядно убедиться в значительном ускорении операций выборки при использовании индексов. Изучены графическое представление планов выполнения запросов во вкладке Explain и история запросов в Query Tool pgAdmin 4. Полученные знания и навыки позволяют эффективно проектировать и оптимизировать запросы к базе данных, обеспечивая высокую производительность и удобство работы с данными.